

10.1 Wewnątrz kontenerów działają proste web serwery, które pobierają input od użytkownika i wyświetlają go. Jeden z serwerów posiada zabezpieczenia przed atakiem XSS, drugi nie.

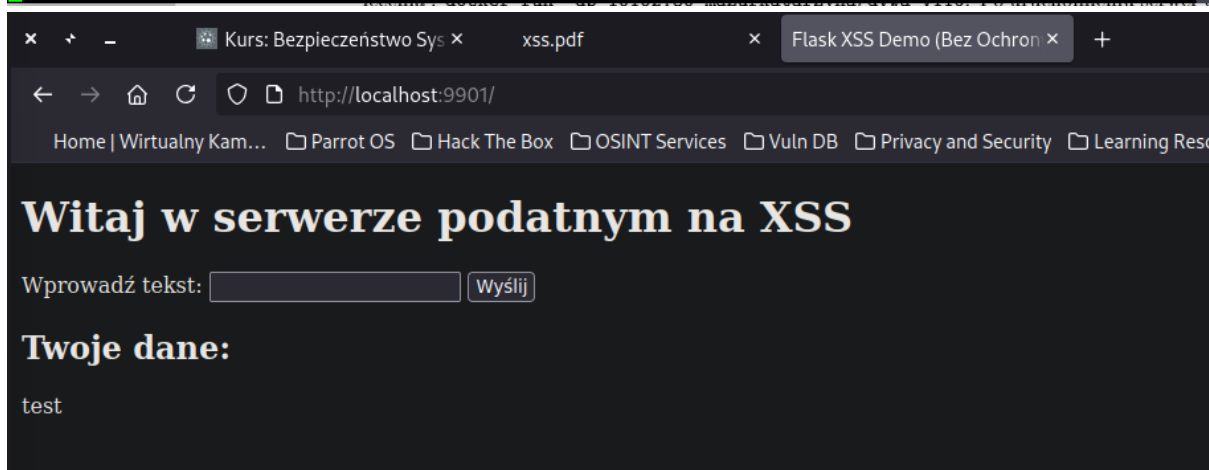
- (a) Korzystając z przygotowanego obrazu Dockerowego `mazurkatarzyna/xss-example-server-1:latest`, spróbuj wykonać atak XSS. Spróbuj wyświetlić zawartość nagłówka `Cookie`. Sprawdź kod źródłowy serwera. W jaki sposób można zmodyfikować kod, aby przeprowadzenie ataku było niemożliwe?

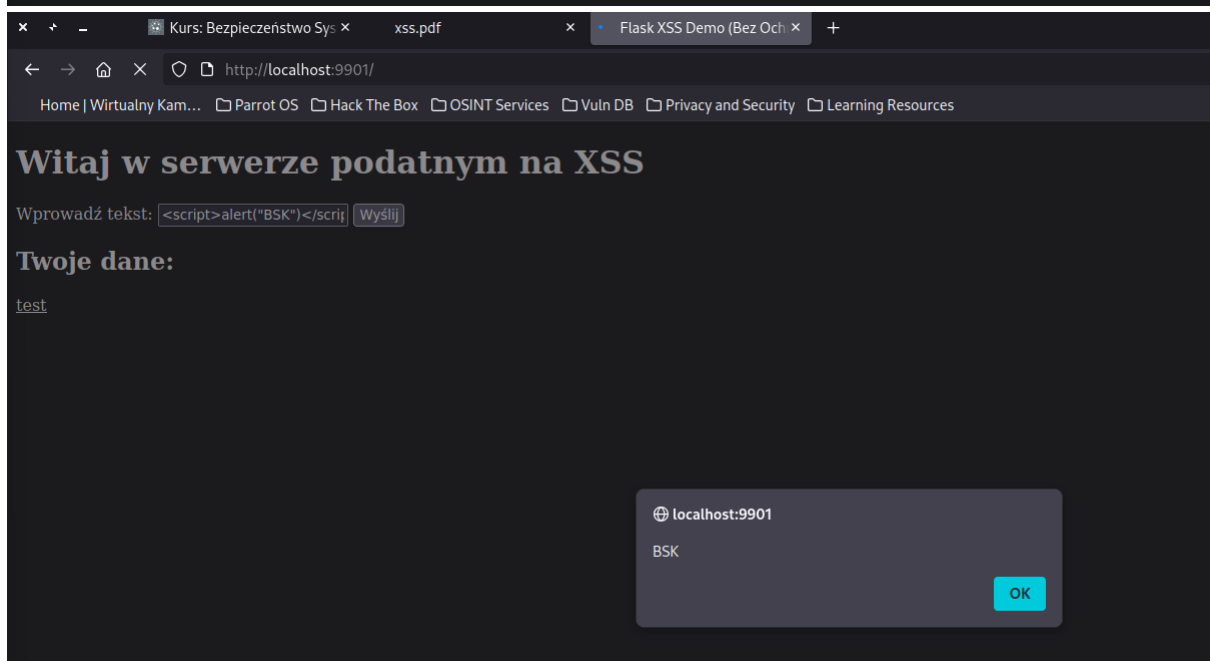
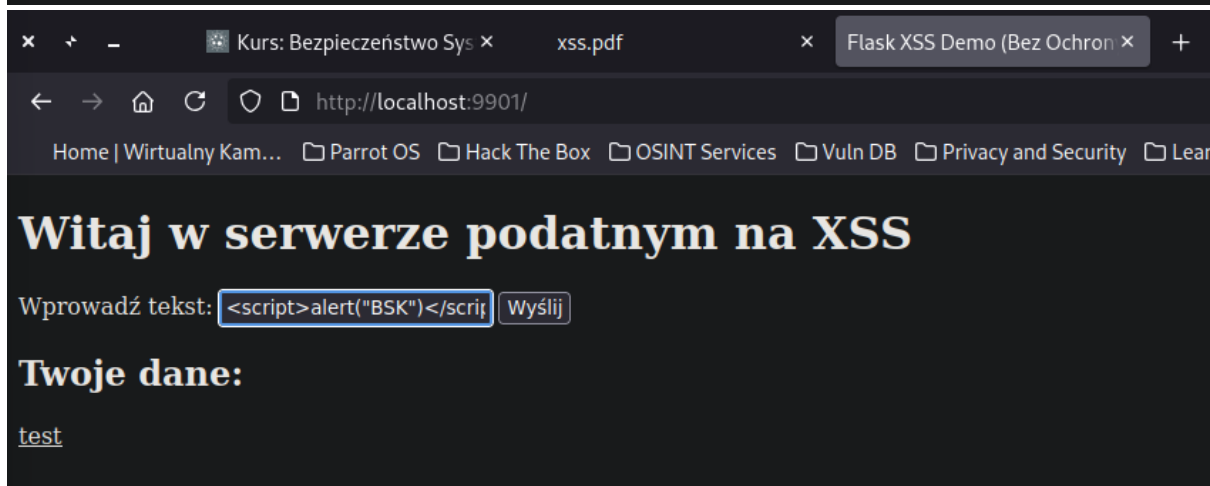
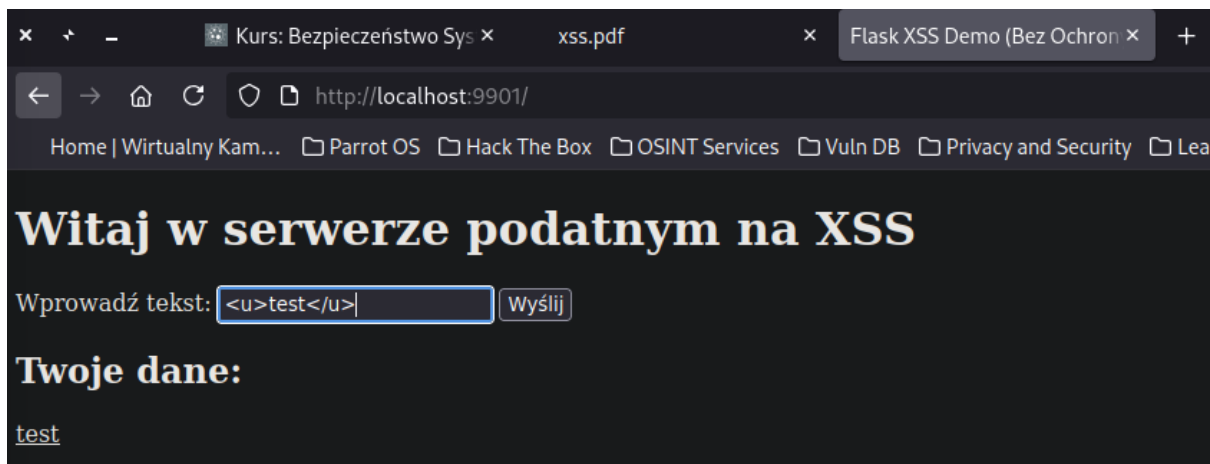
Kontener uruchom za pomocą polecenia: `docker run -it -p 9901:9901 mazurkatarzyna/xss-example-server-1`. Serwer (już wewnątrz kontenera) uruchom za pomocą polecenia `python3 app.py` gdzie `app.py` jest nazwą pliku z serwerem. Po uruchomieniu, serwer będzie działał pod adresem: `http://127.0.0.1:9901`.

- (b) Korzystając z przygotowanego obrazu Dockerowego `mazurkatarzyna/xss-example-server-2:latest`, spróbuj wykonać atak XSS. Spróbuj wyświetlić zawartość nagłówka `Cookie`. Sprawdź kod źródłowy serwera. Czy atak XSS się wykonał? Dlaczego?

Kontener uruchom za pomocą polecenia: `docker run -it -p 9902:9902 mazurkatarzyna/xss-example-server-2`. Serwer (już wewnątrz kontenera) uruchom za pomocą polecenia `python3 app.py` gdzie `app.py` jest nazwą pliku z serwerem. Po uruchomieniu, serwer będzie działał pod adresem: `http://127.0.0.1:9902`.

```
[x]-[user@parrot]-[~]
$ sudo docker run --security-opt apparmor=unconfined -it -p 9901:9901 mazurkatarzyna/xss-example-server-1
root@618c3f56ca83:/app# py
bash: py: command not found
root@618c3f56ca83:/app# ls
no-csp-xss-server.py
root@618c3f56ca83:/app# python3 no-csp-xss-server.py
* Serving Flask app 'no-csp-xss-server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:9901
* Running on http://172.17.0.2:9901
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 115-319-200
```







10.2 Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/dvwa-v110), uruchom serwer za pomocą polecenia: `docker run -dp 10102:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 10102, sprawdź w przeglądarce: `http://127.0.0.1:10102`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność XSS. Podpowiedź: w tym zadaniu wykorzystaj Reflected XSS.

Na poziomie **low security** (brak zabezpieczeń):

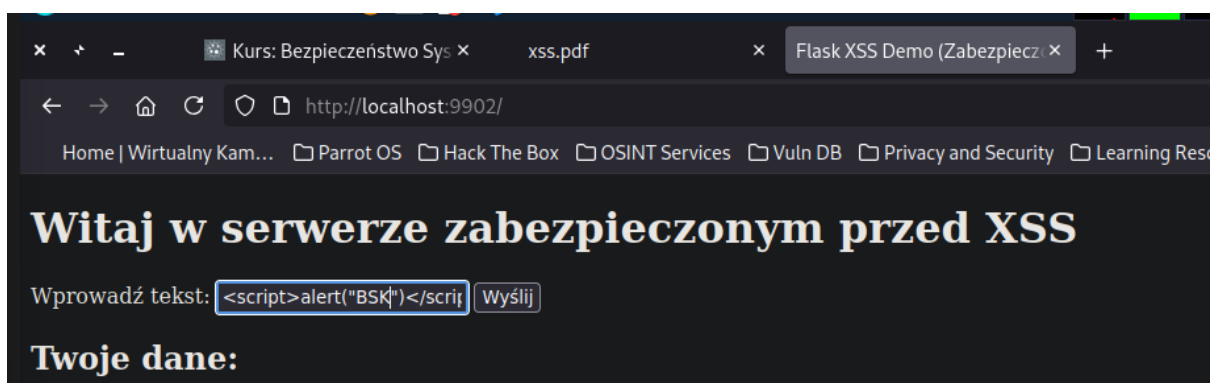
- Ustaw poziom zabezpieczeń na **Low**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS wyświetl dowolną wiadomość.
- Za pomocą ataku XSS wyświetl ciasteczko użytkownika.

Na poziomie **medium security** (proste zabezpieczenia):

- Ustaw poziom zabezpieczeń na **Medium**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS wyświetl dowolną wiadomość.
- Za pomocą ataku XSS wyświetl ciasteczko użytkownika.

Na poziomie **high security** (lepsze zabezpieczenia):

- Ustaw poziom zabezpieczeń na **High**. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS wyświetl dowolną wiadomość.
- Za pomocą ataku XSS wyświetl ciasteczko użytkownika.



← → 🏠 ↻ 📄 http://localhost:9902/

Home | Wirtualny Kam... 📁 Parrot OS 📁 Hack The Box 📁 OSINT Services 📁 Vuln DB 📁 Privacy and Security 📁 Learning Resources

Witaj w serwerze zabezpieczonym przed XSS

Wprowadź tekst: Wyślij

Twoje dane:

<script>alert("BSK")</script>

```
# Function to generate a secure random string (for cookie value)
def generate_random_cookie_value():
    return secrets.token_hex(16) # Generates a random 32-character hex string

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def home():
    user_input = request.form.get("user_input", "")

    # Sanitacja danych wejściowych
    sanitized_input = html.escape(user_input)

    # Generate a random cookie value and set it in the response
    random_cookie_value = generate_random_cookie_value()
```

10.3 Korzystając z przygotowanego obrazu Dockerowego (mazurkatarzyna/dvwa-v110), uruchom serwer za pomocą polecenia : `docker run -dp 10103:80 mazurkatarzyna/dvwa-v110`. Po uruchomieniu serwer działa na porcie 10103, sprawdź w przeglądarce: `http://127.0.0.1:10103`. Serwer to w rzeczywistości web aplikacja, która zawiera wiele podatności, w tym podatność XSS. Podpowiedź: w tym zadaniu wykorzystaj Stored XSS.

Na poziomie `low security` (brak zabezpieczeń):

- Ustaw poziom zabezpieczeń na `Low`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu stored wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu stored wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu stored wyświetl nazwę bieżącej domeny.

Na poziomie `medium security` (proste zabezpieczenia):

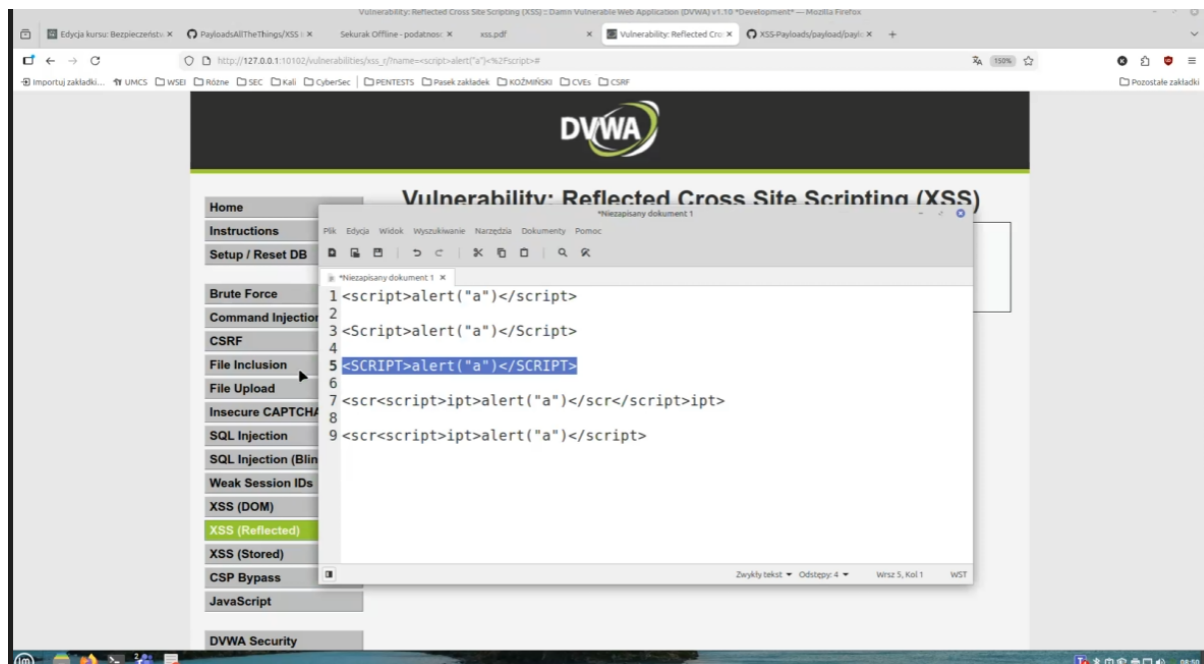
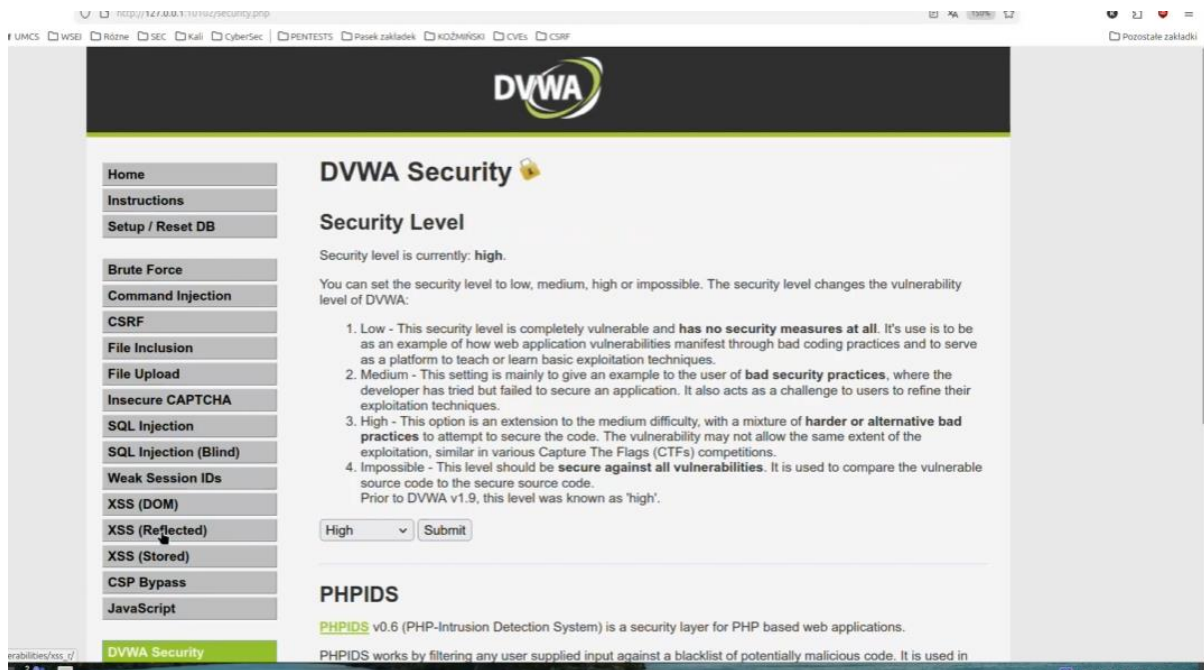
- Ustaw poziom zabezpieczeń na `Medium`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu stored wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu stored wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu stored wyświetl nazwę bieżącej domeny.

9 CROSS SITE SCRIPTING (XSS)

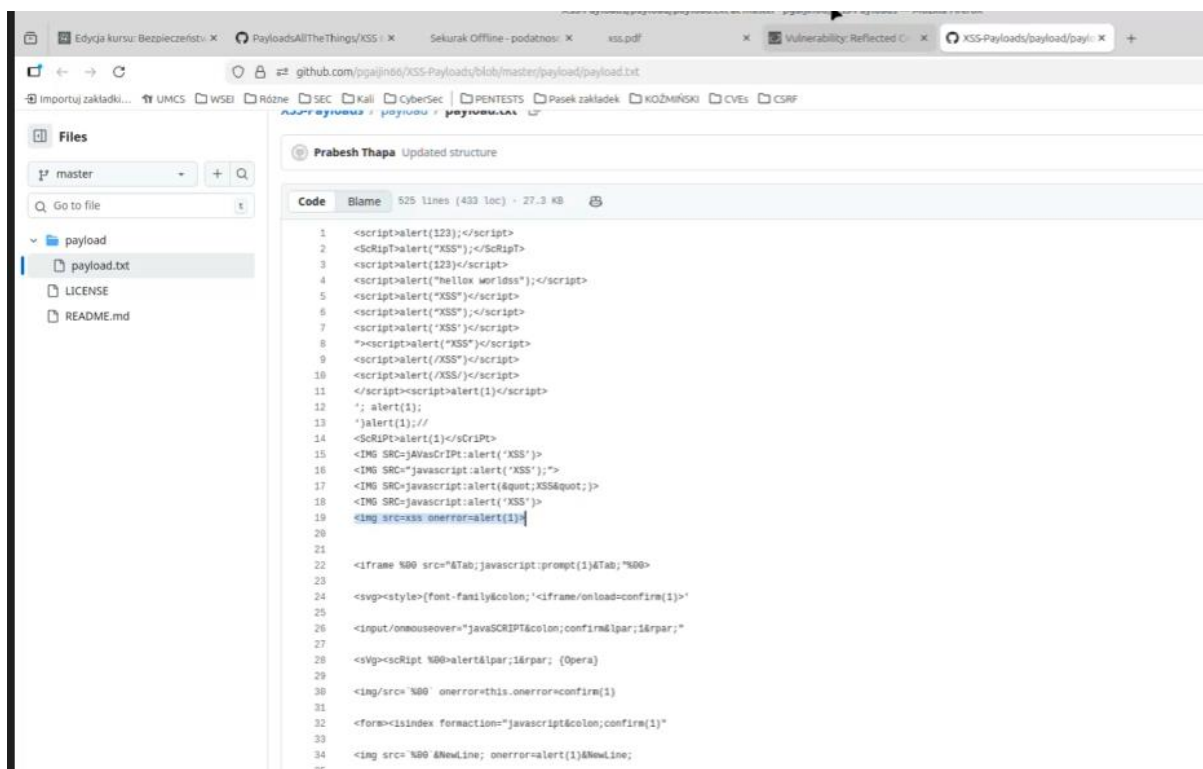
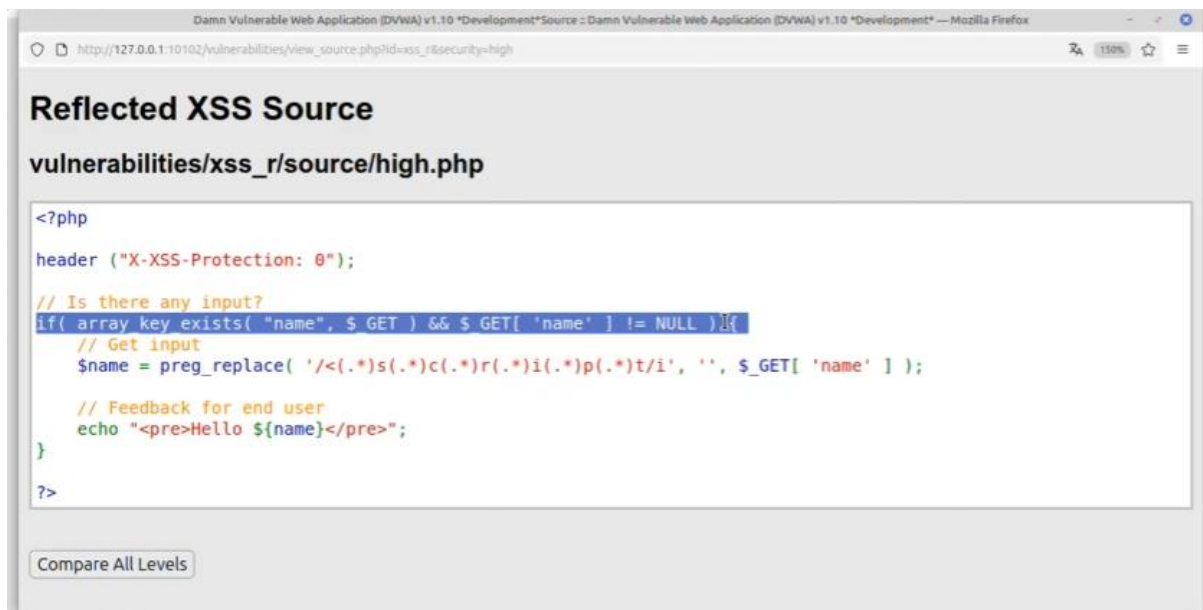
Na poziomie `high security` (lepsze zabezpieczenia):

- Ustaw poziom zabezpieczeń na `Low`. Przeanalizuj kod strony, dlaczego atak jest możliwy?
- Za pomocą ataku XSS typu stored wyświetl dowolną wiadomość.
- Za pomocą ataku XSS typu stored wyświetl ciasteczko użytkownika.
- Za pomocą ataku XSS typu stored wyświetl nazwę bieżącej domeny.





Te nie działają /\



Ten działa /\